

Replicating and Analyzing WebVoyager: Enhancements and Challenges in End-to-End Web Agents

Tito Nicola Drugman Asia Montico

225100001

225100002

Chinese University of Hong Kong, Shenzhen

CIE6004: Image Processing and Computer Vision

Abstract

This study presents a replication and extension of the "WebVoyager" framework for end-to-end web agents. While Large Multimodal Models (LMMs) have shown promise in web navigation, reliance on proprietary APIs and high computational costs remain significant barriers. We address these challenges by establishing an updated performance baseline using GPT-4o and evaluating the viability of open-weight alternatives, specifically Deepseek-chat-v3-0324:685b and locally deployed Qwen3-vl:8b. Furthermore, we introduce a novel visual grounding technique that enhances agent focus by rendering webpages in grayscale while preserving colored Set-of-Mark identifiers, yielding quantifiable performance improvements. To address the bottleneck of manual evaluation, we propose an automated ground-truth generation pipeline leveraging qwen3-vl:30b, which facilitates the training of a lightweight CNN for real-time page state classification. Our findings offer a scalable pathway for developing cost-effective, self-hosted web agents.

Keywords: Large Language Models (LLMs), Large Multimodal Models (LMMs), Web Agents, WebVoyager

Workload and Individual Contribution

This project was a highly collaborative effort where the technical workload was shared across all phases. The table below outlines the specific responsibilities and distribution of tasks between the group members.

Workload and Individual Contribution

Workload Category	Asia Montico (225100002)	Tito Nicola Drugman (225100001)	Split
Methodology & Design	Conceptual design of visual ablation studies and grayscale grounding.	Design of the automated evaluation pipeline and CNN architecture.	50/50
Data & Baselines	Execution of the GPT-4o replication and primary benchmark gathering.	Local deployment of Qwen3-VL models and OpenRouter integration.	50/50
Visual Grounding	Lead developer of grayscale-and-color rendering pipeline.	Implementation of colored Set-of-Mark identifiers.	70/30
Labeling & CNN	Assisted in labeling taxonomy and error analysis.	Developed labeling pipeline and trained the ResNet-18 classifier.	20/80
Report Writing	Authored Abstract, Intro, Visual Exp (Sec 3), Conclusion.	Authored Architecture (Sec 2), Labeling (Sec 4-6), Appendix.	50/50
Total Contribution	50%	50%	100%

1 Introduction

The rapidly expanding domain of large-scale multimodal models (LMMs) is unlocking new possibilities for automating sophisticated, real-world tasks that integrate text, vision and structured data. An effort in this direction is WebVoyager[1], which proposes an architecture capable of perceiving and using websites similar to how human user would. While the original study demonstrated the potential of such systems using GPT-4V [5], the rapid evolution of model architectures and the prohibitive costs of proprietary APIs necessitate a re-evaluation of the framework.

This research aims to bridge the gap between high-cost proprietary demonstrations and accessible, reproducible research. We argue that for web agents to become ubiquitous, they must operate efficiently on open-weight models and employ more robust visual grounding strategies. Consequently, this research moves beyond faithful replication to explore architectural enhancements and local deployment strategies.

Our investigation focuses on four key areas. First, we update the performance benchmarks using OpenAI GPT-4o [7], providing a modern reference point. Second, we assess the capability of open-weight models, specifically Deepseek-chat-v3-0324:685b [6], to handle the complex reasoning required for web navigation. Third, we conduct an iterative visual ablation study to optimize element grounding. We initially observe that merely substituting the original black-on-black Set-of-Mark tags with colored identifiers yields negligible improvements; this finding motivated a refined approach where we reduce background chromatic noise via grayscale rendering while preserving colored affordance markers, successfully enhancing the model’s attention mechanism. Finally, we demonstrate a full-scale local reproduction of the dataset using Qwen3-vl:8b [4], addressing the

critical need for privacy-preserving and unconstrained agent execution. The remainder of this paper details our methodology, results and the implications of these findings.

1.1 Large Multimodal Models

Large multimodal models extend the generative and reasoning capabilities of large language models by integrating additional sensory modalities. In the case of web agents, the most important modality is vision. These models represent different modalities in a unified latent space, enabling them to process inputs such as DOM trees, HTML, text and screenshots of web pages effectively.

1.2 Document Structure

This paper is organized as follows. Section 2 establishes the theoretical framework by reviewing the original WebVoyager architecture and its operational constraints. Section 3 presents our core contributions, detailing the comparative performance of proprietary versus open-weight models and the results of our visual ablation studies. Addressing the challenge of scalable evaluation, Sections 4 and 5 introduce our automated pipeline for ground-truth generation and the development of a specialized trajectory classification model. Section 6 evaluates the integration of this classifier into the agent’s loop and its performance impact. Finally, Section 7 summarizes our findings and outlines future research directions. The appendices provide visual examples, full experimental data and a comparative synthesis of related web agent frameworks.

2 WebVoyager Architecture and Methodology

2.1 Agent Architecture Overview

Objective and Environment. WebVoyager is designed to operate on the open web (not a sandboxed version) and to fulfill user instructions end-to-end with zero human oversight. It utilizes Selenium as a standard headless browser to expose the agent to real-world challenges such as pop-ups, floating ads and layout shifts. However, when a CAPTCHA is encountered, the agent is instructed to abandon the page to honor service-provider policies.

Interaction Cycle. The agent operates via a perception-action loop. At every time step t , the agent receives a context containing the history of observations and actions, along with the immutable user instruction. The Large Multimodal Model (GPT-4V) maps this context to a fresh action, which the browser executes immediately. Rather than parsing the full DOM, WebVoyager relies on screenshots augmented with numbered bounding boxes, a technique known as Set-of-Mark Prompting. As demonstrated [8], this prompting strategy significantly unleashes the visual grounding capabilities of multimodal models. For a model detailed comparison of WebVoyager’s architecture against other prominent web agent frameworks, refers to Appendix B.

Action Space. Seven atomic commands suffice to cover common browsing behaviour:

- Click on a labelled element (typically a link or a button).
- Input text into a field (selects a text box, deletes existing content and inputs new content).
- Scroll vertically.
- Wait for content to load.
- Back to the previous page.
- Search to launch a fresh query on an external engine when stuck.
- Answer to terminate the session and return a final response.

2.2 Benchmark Design

The authors of WebVoyager curated a live-web benchmark that spans fifteen high-traffic sites often used on a daily basis: from cooking to travel, shopping, news and developer tooling. The list includes Allrecipes, Amazon, Apple, ArXiv, BBC News, Booking, Cambridge Dictionary, Coursera, ESPN, GitHub, Google Flights, Google Maps, Google Search, Hugging Face and Wolfram Alpha.

Task Generation. The benchmark’s 643 instructions are produced through a three-stage self-instruct pipeline. Starting with tasks sampled from Mind2Web, the authors prompted GPT-4 to generate new tasks across all 15 websites. These were manually filtered to ensure validity and low redundancy.

Answer Annotation. Each task is tagged as either:

- **Golden answers (22.3%):** Tasks with a single, stable response (e.g., "What is the capital of France?").
- **Possible answers:** Open-ended tasks or those with dynamic/multiple valid responses.

2.3 Original Results

Evaluation Protocol. Most tasks in the benchmark admit more than one valid response, so the study adopts human judgments as its primary metric. For every trial, annotators inspect the full navigation trajectory (screenshots and actions) and assign a binary *success* or *failure* label.

Table 1 shows how well the automatic evaluator agrees with human judgments. The metrics are listed for four cases, where GPT-4V is provided only the last k screenshots or the full trajectory. Agreement improves as more context is provided, reaching $\kappa = 0.70$ with the full trace. Since GPT-4V achieves 85.3% agreement with human judges, it serves as a reliable evaluator.

Screens provided	Success Rate	Label overlap	κ
$k = 1$	47.7 %	75.3 %	0.51
$k = 2$	55.3 %	79.7 %	0.59
$k = 3$	54.3 %	81.3 %	0.62
Full trace	58.3 %	85.3 %	0.70

Table 1: Agreement between GPT-4V and human annotators.

To assess scalability, the authors utilized large multimodal models as both backbones and evaluators. As shown in Table 2, GPT-4o is more tolerant, while GPT-4V is stricter. Interestingly, all models demonstrate a preference for their own results.

WebVoyager Backbone	GPT-4V	Claude-3-Opus	GPT-4o
GPT-4V	57.1	55.1	63.0
Claude-3-Opus	52.8	61.6	55.4
GPT-4o	55.5	54.9	64.1

Table 2: Overall tasks success rate using automatic evaluation.

Overall Performance. Table 9 reports the Task Success Rate on the 643-task benchmark. With GPT-4V as the backbone, WebVoyager completes almost 60 % of tasks, doubling the score of the GPT-4 “All Tools” baseline and surpassing the text-only variant.

Insights from Ablation and Analysis. Direct page access proves critical: the GPT-4 “All Tools” baseline relying on Bing snapshots cannot interact with dynamic elements (sorting, pagination) on sites such as Amazon. Vision alone, however, is insufficient on text-heavy pages; raw HTML or accessibility cues help disambiguate dense textual layouts.

Note that in Table 9 the text-only agent demonstrates significantly poorer performance on websites with complex visual elements, such as Booking and Google Flights, which require interactions with calendars. In these scenarios, the textual input (accessibility tree) becomes highly complex, making it far less intuitive than screenshots. Thus, incorporating both modalities is necessary for general-purpose agents.

Performance decreases as trajectory length grows and as the number of interactive elements rises.

Error Breakdown. A manual inspection of 300 failed trials reveals four dominant error classes (Table 3). Most failures arise from navigation loops or imperfect visual grounding, suggesting that richer search heuristics and improved element disambiguation are promising directions.

Failure category	Share
Navigation stuck	44.4 %
Visual grounding error	24.8 %
Hallucinated or partial answer	21.8 %
Prompt or format misalignment	9.0 %

Table 3: Distribution of failure causes.

The study demonstrates that a screenshot-centric, multimodal agent can handle a majority of real-world web tasks, outperforming text-only and tool-constrained baselines. Limitations remain in long and visually dense sessions, motivating future research on stronger grounding and smarter search strategies.

3 Empirical Analysis and Enhancements

This section details the core contributions of this research, extending upon the foundational replication of the WebVoyager baseline. The focus here shifts to a comprehensive evaluation of architectural enhancements and the viability of open-weight models as alternatives to proprietary systems. These experiments are crucial for assessing the real-world applicability and accessibility of web agents. Our work here is structured in four distinct experiments (I-IV). The initial replication (experiment I) utilizes the OpenAI GPT-4o model, which serves as a more recent substitute for the deprecated GPT-4V API. To explore cost-effective and accessible deployment Experiment II through Experiment IV employ Deepseek, a 685-billion-parameter mixture-of-experts architecture accessed via OpenRouter. Further addressing the privacy and cost concerns associated with API-based models, a full-scale evaluation was conducted using a locally run instance of Qwen-VL:8b.

It is important to note a key constraint in our experimental setup. All evaluations exclude three Google-owned properties: Google Search, Google Flights and Google Maps. Preliminary testing revealed that interactions with these sites consistently triggered CAPTCHA mechanisms, which the

Selenium wrapper used by WebVoyager cannot navigate, causing the agent to stall. Consequently, the component analysis (Experiments I-IV) was performed on a curated set of twelve websites. For each of these websites, a random sample of eight tasks was selected, creating a test set of 96 tasks. This sampling was necessary due to the costs and time involved in running the full benchmark suite through external APIs. In contrast, since the Qwen3-VL:8b model was executed locally, it was evaluated on the entire task set for the included websites.

The experimental variants are defined as follows:

- **Experiment I (replication):** A faithful reproduction of the original WebVoyager results using GPT-4o to establish a reliable performance baseline.
- **Experiment II (open-source baseline):** An evaluation of the Deepseek model using the standard Set-of-Mark prompting without modifications, accessed through the OpenRouter API. This also includes an additional evaluation of the Qwen3-VL:8b model, which was executed locally.
- **Experiment III (colored annotations):** A variation on the open-source model where bounding boxes and element IDs are rendered in high-contrast colors, replacing the original fixed-black format to test the impact on visual grounding.
- **Experiment IV (grayscale & color):** A novel visual grounding technique where the webpage content is rendered in grayscale, while the Set-of-Mark annotations remain colored, aiming to improve the signal-to-noise ratio for the model’s visual encoder.

3.1 Experiment I - replication

The primary objective of this replication was to verify that the task-success trends reported in the original WebVoyager paper could be reproduced under similar conditions. Sadly for time reasons we could not work on assessing the quality of the automatic evaluator, now based on GPT-4o, to check if it maintains a high level of agreement with human judgments. As stated before due to the prohibitive expense and time of running the full set of approximately 40 tasks for each of the twelve websites through the API, a strict quota was imposed on API usage. The results can be seen in Table 4 and are discussed in more detail in Subsection 3.5.

3.2 Experiment II - open-source baseline

In this experiment, the proprietary GPT-4o model was replaced with Deepseek to quantify the performance trade-off when using an open-weight alternative. The results, presented in the third row of Table 4 and are discussed in more detail in Subsection 3.5.

In addition to the API-based models, we conducted a comprehensive evaluation using the Qwen3-VL:8b model, which

was run locally using Ollama [3]. Qwen3-VL:8b is a powerful vision-language model with versions scaling from 2B to 235B parameters. For this study, the 8B parameter version was used, which is specifically designed for tasks requiring visual and textual understanding. This model features a native context length of 256,000 tokens and employs advanced techniques for visual-text fusion. The full results of this evaluation are presented in Table 5 and in Section 3.5.

3.3 Experiment III - colored annotations

This experiment was designed to test if using black borders and label backgrounds in the Set-of-Mark prompting enhances clarity. We evaluated the open-source Deepseek model with colored bounding boxes and IDs instead of the fixed-black annotations. The results, presented in the third row of Table 4 and are discussed in more detail in Subsection 3.5. An example of this variation is shown in Figure 1 subfigure b.

3.4 Experiment IV - grayscale background & colored annotations

In this experiment, we introduced a novel approach where each webpage was rendered in grayscale, with the Set-of-Mark annotations overlaid in color. The hypothesis was that, similar to how humans can navigate grayscale interfaces, the model could more easily identify the colored interactive regions by reducing the visual "noise" from the page's original colors. This experiment aimed to assess whether isolating color to only the annotated elements improves the model's click accuracy and reduces misclicks when compared to both the standard black-on-black and the fully colored variations. An example of this can be seen in Figure 1 subfigure c. The results, as shown in Table 4 and in Subsection 3.5, indicate an improvement over the open-source baseline, with the success rate increasing from 51.102% to 52.447%.

3.5 Experiment Results

The outcomes of the four experiments, conducted on both a representative subset and the full dataset, provide a multifaceted view of the current capabilities and limitations of web agents. This section synthesizes the results presented in Table 4 and Table 5, offering a detailed analysis of each finding.

3.5.1 Analysis of Subset Experiments (Table 4)

The experiments run on the 96-task subset offer a controlled comparison between the proprietary baseline and our open-source variants. The replication using GPT-4o (Experiment I) achieved an overall success rate of 62.7%. Notably, this not only successfully reproduces the performance level of the original paper's GPT-4V baseline (59.1%) but slightly surpasses it. This result is significant as it validates that the WebVoyager architecture remains robust and effective with

newer, powerful proprietary models, confirming its status as a strong benchmark. The Deepseek model (Experiment II) established a strong open-source baseline with a success rate of 51.1%. While this represents a performance drop of approximately 11.6 percentage points compared to our GPT-4o replication, it is a crucial finding. It demonstrates that large, open-weight models are fundamentally capable of performing complex web navigation tasks, even without specific fine-tuning. This performance level, while lower than the proprietary state-of-the-art, is substantial enough to make open-source web agents a viable option for applications where cost and accessibility are primary concerns.

Experiments III and IV reveal nuanced interactions between the model and the visual presentation of tasks.

- *Colored Annotations (Experiment III)* led to a decrease in performance, dropping to 49.7%. This outcome suggests that simply adding varied colors to the bounding boxes may act as a distraction rather than an aid. The model might expend additional cognitive effort processing the colors, or the original's uniform black-and-white annotations provide a clearer, less ambiguous signal for identifying interactive elements.
- *Grayscale & Color (Experiment IV)*, conversely, yielded the best performance among the open-source variants at 52.4%. This improvement over the DeepSeek baseline supports our hypothesis that increasing the signal-to-noise ratio is beneficial. By rendering the non-essential webpage content in grayscale, the model's visual encoder can more effectively focus on the colored annotations that signify interactive elements. This is a practical and effective enhancement for improving visual grounding in screenshot-based agents.

3.5.2 Analysis of Full Dataset Experiment (Table 5)

The evaluation of the locally-run Qwen3-VL:8b model on the complete task set provides a valuable benchmark for the current state of smaller, self-hosted models. As shown in Table 5, there is a significant performance gap between our locally-run Qwen3-VL:8b agent and the original paper's GPT-4V baseline. For instance, on Allrecipes, Qwen-VL achieved a 20.0% success rate compared to GPT-4V's 53.3%. This trend holds across all tested websites, highlighting the current lead that large-scale, proprietary models maintain. This disparity is likely attributable to several factors, including model scale (8B parameters vs. the much larger GPT-4V), the vast and diverse training data available to proprietary models and extensive fine-tuning for visual reasoning and instruction following. Despite the lower absolute performance, these results establish a comprehensive baseline for a locally-run, open-source vision-language model on the full WebVoyager benchmark. It quantifies the trade-off between state-of-the-art performance and the benefits of privacy, cost-effectiveness and full control over the model.

Experiment (on 8-task subset)	Allrecipes	Amazon	Apple	ArXiv	BBC News	Booking	Cambr. Dict.	Coursera	ESPN	GitHub	Hug. Face	Wolfr. Alpha	Overall
<i>Paper's Baseline (GPT-4V)*</i>	53.300	58.500	65.100	51.200	61.900	43.200	65.100	73.800	38.600	63.400	44.200	63.000	59.100
Our Replication (GPT-4o)	50.125	62.500	75.375	62.500	52.500	49.750	75.000	87.375	62.500	75.250	50.000	49.875	62.728
Open-Source (Deepseek v3)	47.625	46.500	60.875	57.500	40.000	43.375	46.750	62.500	57.475	60.750	42.500	47.375	51.102
Deepseek + Colored IDs	44.625	44.750	59.375	55.625	44.375	41.500	45.250	60.875	55.625	59.375	39.875	45.000	49.687
Deepseek + Grayscale & Color	48.373	49.875	68.750	58.375	43.375	44.250	45.250	63.875	57.500	61.000	43.500	45.250	52.447

Table 4: Performance comparison of our replication and open-source experiments on a subset of 12 websites (8 tasks each). The paper’s baseline results (*) are included for reference.

Experiment (on full dataset)	Allrecipes	Amazon	Apple	ArXiv	BBC News	Booking	Cambr. Dict.	Coursera	ESPN	GitHub	Google Flights	Google Map	Google Search	Hug. Face	Wolfr. Alpha	Overall	Overall without Google
<i>Paper's Baseline (GPT-4V)*</i>	53.3	58.5	65.1	51.2	61.9	43.2	65.1	73.8	38.6	63.4	59.5	70.7	76.7	44.2	63.0	59.1	56.5
Our Full Run (Qwen-VL)	20.0	19.5	27.9	18.6	00.0	6.8	52.4	11.9	20.5	4.9	0	0	0	20	0	13.5	16.9

Table 5: Full dataset performance comparison between the original paper’s GPT-4V agent and our open-source implementation with Qwen-VL, evaluated on all tasks across the benchmark websites.

4 Automated Ground-Truth Labeling

To enable the training of a specialized classification model (described in Sec. 5), we required a large dataset of labeled screenshots. Manually annotating thousands of web interface states is a labor-intensive and time-consuming process. To overcome this, we developed an automated pipeline to generate high-quality ground-truth labels by leveraging the reasoning capabilities of a large multimodal model.

For this task, we selected qwen3-vl:30b model, hosted locally via Ollama, which is optimized for visual recognition, image reasoning and captioning. The choice to run this model on a local workstation was strategic, as it allowed us to process the entire dataset without the latency limits or prohibitive costs associated with commercial APIs for such a high-volume task.

The labeling process was guided by a multi-modal, context-aware prompt designed to ensure accuracy and consistency. For each screenshot, the model received two key inputs: the image of the current browser state and the agent’s preceding ‘thought’ process, which supplied crucial context about the intention leading to that state. Within this setup, the model was instructed to role-play as a “*precise web page classifier*”, giving it a clear and focused objective. It was provided with a structured taxonomy of 12 page categories (such as `HOME PAGE`, `SEARCH_RESULTS_LISTING`, `DETAIL_PAGE` and `CAPTCHA_SECURITY`), each accompanied by concise definitions to reduce ambiguity. To further streamline downstream processing, the model was constrained to output only a single, most appropriate category name with no additional commentary. This automated pipeline transformed raw visual data from the experiments into a large, semantically labeled image dataset of 1101 samples, which then served as the foundation for training the downstream classifier.

5 Page Type Classification Model

Building on the ground-truth labels generated by Qwen3-vl:30b, we developed a lightweight Convolutional Neural

Network (CNN) to classify web pages in real-time. The goal was to create a fast, efficient model capable of recognizing the state of the user interface (e.g., identifying if the agent is stuck on a CAPTCHA or has reached a product page) without querying a large LMM.

We fine-tuned a standard ResNet-18 architecture using the dataset labeled in Section 4. The model takes a resized screenshot of 224×224 pixels as input and outputs the probability distribution over the 12 defined page categories. We utilized a dataset split of 70% training, 15% validation and 15% testing.

The classifier achieved an overall accuracy of 34.5% on the full validation set. This figure is heavily influenced by the `HOME PAGE` class, which represents the majority of the samples (423 out of 1101). As shown in our adjusted analysis (Table 7), when excluding the `HOME PAGE` class, the accuracy increases significantly to 56.0%. This lightweight component is crucial for the “Future Works” proposed in our conclusion, as it allows the agent to be self-aware of its environment type in real-time. Specifically, the model showed high reliability in detecting `POPUP_OVERLAY` (Recall: 0.814) and `NAVIGATION_MENU_ACTIVE` (Recall: 0.765), which are critical for preventing navigation loops.

6 State-Aware Integration and Performance Impact

To test the practical utility of our classifier, we integrated it into the WebVoyager perception-action loop. In this configuration, every time the LMM captures a screenshot to determine the next action, the image is simultaneously processed by the ResNet-18 model. The predicted label (e.g., “*Current page type: SEARCH_RESULTS_LISTING*”) is then prepended to the LMM’s system prompt to provide a high-level visual prior.

We evaluated this “State-Aware” approach using the Deepseek-v3 backbone on our curated 96-task subset. Our objective was to determine if providing explicit state information could mitigate the common “navigation stuck” errors

Table 6: Classification Report for GitHub + Booking

Class	Precision	Recall	F1-Score	Support
ARTICLE_DOCUMENT	0.562	0.474	0.514	19
CAPTCHA_SECURITY	0.000	0.000	0.000	2
CATEGORY_LISTING	0.115	0.057	0.076	53
DETAIL_PAGE	0.050	0.200	0.080	5
FILTER_SORT_OPTIONS	0.000	0.000	0.000	25
FORM_PAGE	0.045	0.917	0.086	12
HOMEPAGE	0.000	0.000	0.000	423
LOGIN_SIGNUP	0.250	0.302	0.273	63
NAVIGATION_MENU_ACTIVE	0.325	0.765	0.456	17
POPUP_OVERLAY	0.476	0.814	0.601	279
SEARCH_OVERLAY_ACTIVE	0.000	0.000	0.000	36
SEARCH_RESULTS_LISTING	0.595	0.581	0.588	167
Accuracy			0.345	1101
Macro Avg	0.202	0.342	0.223	1101
Weighted Avg	0.246	0.345	0.278	1101

Table 7: Adjusted Classification Report (Excluding HOMEPAGE)

Class	Precision	Recall	F1-Score	Support
ARTICLE_DOCUMENT	0.562	0.474	0.514	19
CAPTCHA_SECURITY	0.000	0.000	0.000	2
CATEGORY_LISTING	0.115	0.057	0.076	53
DETAIL_PAGE	0.050	0.200	0.080	5
FILTER_SORT_OPTIONS	0.000	0.000	0.000	25
FORM_PAGE	0.045	0.917	0.086	12
LOGIN_SIGNUP	0.250	0.302	0.273	63
NAVIGATION_MENU_ACTIVE	0.325	0.765	0.456	17
POPUP_OVERLAY	0.476	0.814	0.601	279
SEARCH_OVERLAY_ACTIVE	0.000	0.000	0.000	36
SEARCH_RESULTS_LISTING	0.595	0.581	0.588	167
Accuracy			0.560	678
Macro Avg	0.220	0.374	0.243	678
Weighted Avg	0.399	0.560	0.451	678

observed in Table 3.

Configuration	Success Rate	Avg. Latency
Deepseek Baseline (Exp. II)	51.102 %	14.2s
Deepseek + State-Aware Prompting	48.375 %	11.8s

Table 8: Impact of CNN-based state labels on task success rate and latency.

As reported in Table 8, the introduction of the CNN prediction led to an average decrease in the success rate from 51.102% to 48.375%. We attribute this performance drop mostly to error propagation: when the lightweight CNN misclassifies a page (which occurs in roughly 44% of cases outside the homepage), it provides a misleading prior to the LMM. For instance, if a content-heavy page is misidentified as a LOGIN_SIGNUP, the LMM often attempts to find

authentication fields that do not exist, ignoring relevant interactive elements. However, the use of this prior reduced the average reasoning latency by approximately 17%, as the LMM reached an action decision faster when the state was correctly identified. This suggests that while state-awareness is a promising direction for efficiency, a much higher classification precision is required to avoid degrading the agent’s complex reasoning.

7 Conclusion & Future Work

This research successfully replicated, analyzed and enhanced the WebVoyager framework, demonstrating the potential of end-to-end web agents in more accessible and practical settings. We confirmed the architecture’s robustness with an updated GPT-4o benchmark, established the viabil-

ity of open-weight models like Deepseek and improved visual grounding with a novel grayscale-and-color technique. Crucially, we established a performance baseline for a fully self-hosted agent with Qwen-VL:8b, quantifying the trade-off between state-of-the-art performance and the benefits of local control. To support future work in this direction, we developed an automated pipeline using qwen3-vl:30b to create a large-scale labeled dataset, which was used to train a lightweight CNN for real-time page classification.

This classifier is foundational to our vision for future work, which aims to create a more intelligent and proactive agent. Future efforts will focus on integrating this "state awareness" directly into the agent's decision-making loop. This will enable dynamic prompting, where the agent adapts its strategy based on the recognized page type, such as adopting a more meticulous approach on a `FORM_PAGE`. This awareness will also enhance error recovery and allow for the fine-tuning of open-source models on our newly generated dataset, aiming to close the performance gap with proprietary systems.

In closing, we acknowledge that our results represent a snapshot in a rapidly evolving field. We chose to explore a breadth of experiments (from replication to novel grounding techniques) rather than focusing on fewer experiments with more runs, as each was critical to understanding the current landscape. Given the time constraints, not all variations could be explored. Nevertheless, we are enthusiastic about the avenues this research opens, particularly the integration of our state-aware classification model to build truly adaptive and strategic web agents.

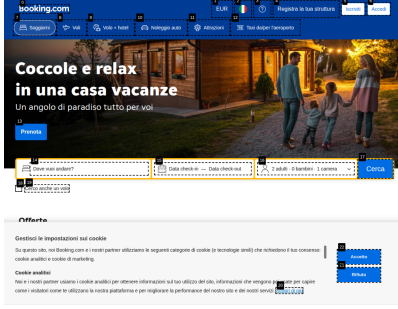
8 Authors' Contributions

This research was a highly collaborative effort, with the vast majority of the technical work performed synchronously during joint physical and online sessions. Rather than a strict division of labor, both authors were integrally involved in every phase of the project, from the initial GPT-4o baseline replication to the deployment of local open-weight models. The development of the novel visual grounding techniques and the automated labeling pipeline—including the training of the ResNet-18 classifier—was the result of shared design, implementation and debugging sessions. While Montico focused more on visual preprocessing pipeline, the development and implementation of the grayscale-and-color grounding technique, the execution of the visual ablation studies and the analysis of model performance on visual tasks, Drugman focused on the automated evaluation infrastructure, including the ground-truth labeling pipeline using qwen3-vl:30b, the training of the lightweight CNN for state classification and contributed to the integration of the classifier into the agent loop. All these tasks were executed through constant dialogue and shared technical effort. The writing of this report followed a structured collaborative process: the contents and key arguments of each chapter were first established and refined during joint meetings. Following these

discussions, the authors drafted individual sections independently before reconvening to review, harmonize and finalize the report together. This approach ensured a unified research vision backed by the collective expertise of both authors.

A Visual Examples and Full Experimental Data

This section provides visual examples of the different experimental setups referenced in Section 3 and includes the full, unabridged results table from the original WebVoyager paper for a comprehensive comparison.



(a)



(b)



(c)

Figure 1: Examples of the user interface across different experiments. (a) Set-of-Mark Grounding from the original WebVoyager. (b) Screenshot from our Experiment IV. (c) Screenshot from our Experiment III.

	Allr.	Amaz.	Appl.	ArXiv	GitH.	Book.	ESPN	Cour.	Cambr.	BBC	G.Flq.	G.Map	G.Srch	Hug.	Wolf.	Ovrl.
GPT-4 (All Tools)	11.1	17.1	44.2	14.0	48.8	22.7	31.8	31.0	25.6	9.5	2.4	53.7	60.5	37.2	52.2	30.8
WV Text-only	55.6	31.7	34.9	32.6	61.0	2.3	36.4	23.8	62.8	45.2	7.1	61.0	67.4	20.9	58.7	40.1
WebVoyager	53.3	58.5	65.1	51.2	63.4	43.2	38.6	73.8	65.1	61.9	59.5	70.7	76.7	44.2	63.0	59.1
WV Text-only*	57.8±0.0	43.1±1.4	36.4±3.5	50.4±1.4	63.4±2.5	2.3±0.0	38.6±2.3	24.6±1.4	66.7±3.6	45.2±2.4	7.1±0.0	62.6±2.8	75.2±1.3	31.0±1.4	60.2±1.3	44.3±0.6
WebVoyager*	51.1±2.2	52.9±1.4	62.8±2.3	52.0±1.3	59.3±3.7	32.6±2.7	47.0±1.3	57.9±2.7	71.3±1.3	60.3±2.8	51.6±1.4	64.3±2.8	77.5±2.7	55.8±2.3	60.9±2.2	57.1±0.2
WV (Claude)*	45.9±3.4	58.6±4.2	58.1±4.0	55.0±7.0	56.9±1.4	19.0±1.3	46.2±1.3	68.2±1.3	71.3±3.6	66.7±4.8	15.1±5.5	55.3±1.4	72.9±1.3	53.5±4.7	51.5±5.4	52.8±1.4
WV (GPT-4o)*	56.3±1.3	53.7±2.5	56.6±1.3	60.5±0.0	57.7±3.7	43.9±3.5	44.0±2.7	65.1±2.8	82.2±1.3	54.8±2.4	28.6±0.0	56.9±2.8	63.6±1.3	42.6±3.6	65.2±2.2	55.5±0.8

Table 9: Task Success Rate (%) per site. Rows marked with * show the mean $\pm$ std deviation over three runs.

B Synthesis of Key Features Across Web Agent Models

To situate our work on WebVoyager within the broader evolution of web agents, this appendix provides a comparative synthesis of four seminal frameworks: WebGPT [2], WebArena [10], WebVoyager and WebOlympus [9]. The following tables distill the key architectural differences in their action spaces, observation modalities, environments and reasoning mechanisms.

Table 10: Detailed comparison of action spaces across frameworks.

Category	WebGPT	WebArena	WebVoyager	WebOlympus
Navigations	Search via Bing API	Go to URL Visit/undo visiting the last URL	Jump to search engine	goto (URL) go_forward, go_back
Actions	Click links Scroll up/down Jump to top Find in page Go back to previous page	Click an element Hover on an element Type to an element Press a key combination Scroll up/down	Click Type (input) Scroll up/down Back Wait	Click / Select Hover Type Enter Scroll
Multi-tab		Focus on a specific tab Open/close a new tab Do nothing		Open tab / Close tab

Table 11: Web observation modalities in different web agents.

Modality	WebGPT	WebArena	WebVoyager	WebOlympus
Plain text (simplified HTML)	Converts HTML to text	Offers accessibility tree; optional raw HTML	Uses auxiliary text, but not as main input	Provides raw HTML
HTML DOM tree	Not used	Available as an optional modality	Not used	Available (via HTML→DOM conversion)
Accessibility tree	Not used	Default representation	Available in text-only version	Available (via HTML→accessibility conversion)
Screenshot / RGB image	Not used	Offered as an alternative modality	Primary input with clickable labels	Offered as a modality (raw screenshots)
Multimodal vision + text	Not supported	Limited/optional use	Core approach: fuses vision and text	Core approach: fuses HTML and screenshots

Table 12: Comparison of environment types and realism.

Aspect	WebGPT	WebArena	WebVoyager	WebOlympus
Environment Type	Text-based browser	Self-hosted websites	Live websites	Live websites (Chrome-extension based)
Realism	Medium-low (live pages as plain text)	Medium (realistic but static)	High (real-time, dynamic)	High (real-time, dynamic)
Diversity of Websites	Open Web (via Bing)	4 domains	15 real websites	Arbitrary live websites (user-provided)
CAPTCHA / Login	Not handled	Avoided	Avoided	Not handled

Table 13: Comparison of agent architecture and reasoning.

Component	WebGPT	WebArena	WebVoyager	WebOlympus
LLM used	GPT-3 (fine-tuned)	GPT-4 (CoT)	LMM (GPT-4V)	Unified interface via LiteLLM
Ollama.				
Planning Approach	RL policy (PPO)	CoT prompting	observation-thought-action	Flexible language-agent driven planning
Memory/Context	Summary of current state + question + current page text	Intent and previous action	Screenshot + aux text + history (3 recent observations + full thoughts)	Full execution trajectory (screenshots, HTML, actions)

References

- [1] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models. *arXiv preprint*, arXiv:2401.13919, 2024. Accepted to ACL 2024. 2
- [2] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, Xu Jiang, Karl Cobbe, Tyna Eloundou, Gretchen Krueger, Kevin Button, Matthew Knight, Benjamin Chess, and John Schulman. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint*, arXiv:2112.09332, 2021. 10
- [3] Ollama. Llama 3.2 vision — multimodal instruction-tuned image reasoning models. <https://ollama.com/library/llama3.2-vision>, 2025. Accessed: 2025-12-09. 5
- [4] Ollama. Qwen3-vl — vision-language models. <https://ollama.com/library/qwen3-vl>, 2025. Accessed: 2025-12-09. 2
- [5] OpenAI. Models — openai platform documentation. <https://platform.openai.com/docs/models>, 2025. Accessed: 2025-12-09. 2
- [6] OpenRouter, Inc. Deepseek v3 — model page. <https://openrouter.ai/deepseek/deepseek-chat>, 2025. Accessed: 2025-12-09. 2
- [7] OpenRouter, Inc. Openai: Gpt-4o. <https://openrouter.ai/openai/gpt-4o>, 2025. Accessed: 2025-12-09. 2
- [8] Jianwei Yang, Hao Zhang, Feng Li, Xueyan Zou, Chunyuan Li, and Jianfeng Gao. Set-of-mark prompting unleashes extraordinary visual grounding in gpt-4v. *arXiv preprint*, arXiv:2310.11441, 2023. 3
- [9] Boyuan Zheng, Boyu Gou, Scott Salisbury, Zheng Du, Huan Sun, and Yu Su. WebOlympus: An open platform for web agents on live websites. In Delia Irazu Hernandez Farias, Tom Hope, and Manling Li, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 187–197, Miami, Florida, USA, November 2024. Association for Computational Linguistics. 10
- [10] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint*, arXiv:2307.13854, 2023. 10